OOP with Java - DB Briefing

COMSM0086

Dr Simon Lock & Dr Sion Hannuna

The "DB" Exercise

Aim of this exercise is to build a database server...
...from the ground up !

A complex application for you to practice your Java
A chance to explore DB content from other units
As well as gain experience using a query language

This exercise does NOT contribute to your unit mark
(In previous years it would have, but not now !)

Overview of Server Operation

Your database server must operate as follows:

1. Receive incoming requests from a client (conforming to a standard query language)
2. Interrogate & manipulate a set of stored records (maintained as a persistent collection of files)
3. Return an appropriate message back to the client (Success or Failure, with data - where relevant)

Data Storage

A database consists of a number of 'tables'
Each 'table' consists of a number of 'columns'
Each 'table' contains 'rows' that store 'records'

The 'tables', their 'columns' and 'record' content
should be stored as TAB separated text files

example-table.txt
example-table.tab

Record IDs

First (0th) column in a table is always called 'id'
Numerical value that uniquely identifies record/row
ID values are automatically generated by the server
The IDs acts as a primary key for each record
Relationships between records use ID as reference
Note that the ID of a record should NEVER change
(or you risk breaking any relationships stored !)
No "recycling" (don't reuse the IDs of deleted rows)

Communication

A minimal skeleton server is provided for you
(so you don't need to worry about networking)

Server listens on port 8888 and passes incoming
messages to the 'handleCommand' method
Your task: implement 'handleCommand' internals !

For simplicity, assume only a single client connects
(i.e. there is no need to handle parallel queries)

Query Language

Clients communicate with server using "simple" SQL:

- USE: changes the database we are querying
- CREATE: constructs a new database or table
- INSERT: adds a new record (row) to a table
- SELECT: searches for records that match a condition
- UPDATE: change existing data contained in a table
- ALTER: add or remove the columns in a table
- DELETE: removes records that match a condition
- DROP: removes a specified table or database
- JOIN: performs an INNER JOIN on two tables

Defining the Query Language

To help specify our simplified query language
We have provided an "augmented" BNF grammar
BNF

We hope that you appreciate this grammar
It embodies much analysis and specification work
It was a significant challenge to create and refine !

There is also a "transcript" of typical queries
Provides examples of queries you might expect to see
transcript

Error Handling

Your parser should identify errors within queries:
- queries that do not conform to the BNF
- queries attempting "illegal" actions (see workbook)

Your response to the client must begin with either:
- [OK] for valid and successful queries
  (followed by the results of the query)
- [ERROR] if there is a problem with the query
  (followed by a human-readable message)

Use exceptions to handle errors *internally*
However these should NOT be returned to the user

Testing

A command-line client has been provided for you
This is really just for demonstration purposes

Development should make use of automated testing
A template test script is found in the Maven project
Add all your automated JUnit tests there as usual

Testing should target the 'handleCommand' method
(Since this is where your code has been inserted)

Importance of Testing

With the advent of Gen AI tools for code creation
Testing is becoming more and more important !
(We need to validate and verify generated code)

It is essential that you improve your testing skills
(This will be assessed in the final CW assignment !)
You had better start working on this ability NOW !

The Software Engineering unit will provide guidance
We will also do an "Effective Testing" session shortly

Questions ?

SQL whitespace variability

As with any programming language (including Java)
SQL can contain extra whitespace and still be valid
Your interpreter needs to be able to cope with this

For example, let's consider this INSERT statement:

```
INSERT INTO marks VALUES('Steve',55,TRUE);
```

How many variants are there with 1 extra space ?

all-possible-variants

generate-variants-script

Dealing with these can be a bit tricky !

Would you like a few extra pointers ?

(Warning: May Contain Metaphors)

How do you chop an onion ?

The "Computer Scientist" Approach

Select the newest, "most cutting edge" knife

Cut onion into two halves (binary chop)

Iterate through columns first (slice)

Iterate through rows next (dice)

After each cut, return chopped

onions into a heap (or stack)

(to keep working area clear)

How does that sound ?

Oh dear me NO !

You need to pull the onion out of the ground first !

Then remove roots (and any remaining soil)
Chop off the green leaves
Remove the outer layers of skin
Maybe even give it a wash
Throw it away if it looks rotten !

Only THEN try to chop it

The Problem

We had assumed that the onion was "ready to go"
But the farmer and seller have done a lot of work...

`Washing, Head and Tailing, Grading, Filtering etc.`

All before you get your hands on the onion

How does this relate to parsing SQL ?!?

Don't just "dive in" and start "slicing and dicing"
Pre-processing will save you a LOT of time & effort

We can do simple filtering / cleaning / scrubbing
In order to "normalise" the incoming commands
Get them into a standard size/shape - like onions !

If everything is similar and consistent...
It will make writing a parser a WHOLE LOT easier

BasicTokeniser